PROJECT PRESENTATION

LIBRARY MANAGEMENT SYSTEM

A Java-based system for managing books, borrowers and library circulation

Java SE • JDBC • Java DB (Derby) • Object-Oriented Design

Based on the public project repository by Haris Muneer

1. Project Overview

A central system to organize common library activities and keep circulation records consistent.

Purpose

- Digitize the basic work of a library.
- Maintain book, person and loan information.
- Support search, issue, return, renewal and hold requests.

Project approach

- Developed as an Object-Oriented Analysis and Design project.
- Keeps interface code minimal and separates domain entities.
- Uses a console-based interface.

Main goal: make library records and circulation tasks easier to manage through a structured Java application.

2. Users and Their Roles

The system defines four main actors, each with a different level of responsibility.

Borrower	Search books; view personal details and currently borrowed books; place a hold on an unavailable title.
Checkout Clerk	Perform borrower tasks, issue and receive books, renew loans, record fine payments, and maintain borrower details.
Librarian	Perform clerk tasks and manage the library collection by adding, deleting or editing book records.
Administrator	Add clerks and librarians; view issued-book history and the complete book collection.

Access is organized around the work each role is permitted to perform.

3. Main Functionalities

The documented use cases cover the main book-circulation lifecycle.

Book catalogue

- Search by title, author or subject.
- Add, update and delete book records (librarian).

Borrowing

- Issue a book to a borrower.
- Record the borrower and issue details.
- Renew an existing loan.

Returns and fines

- Check in returned books.
- Record return details.
- Record when a fine has been paid.

Hold requests

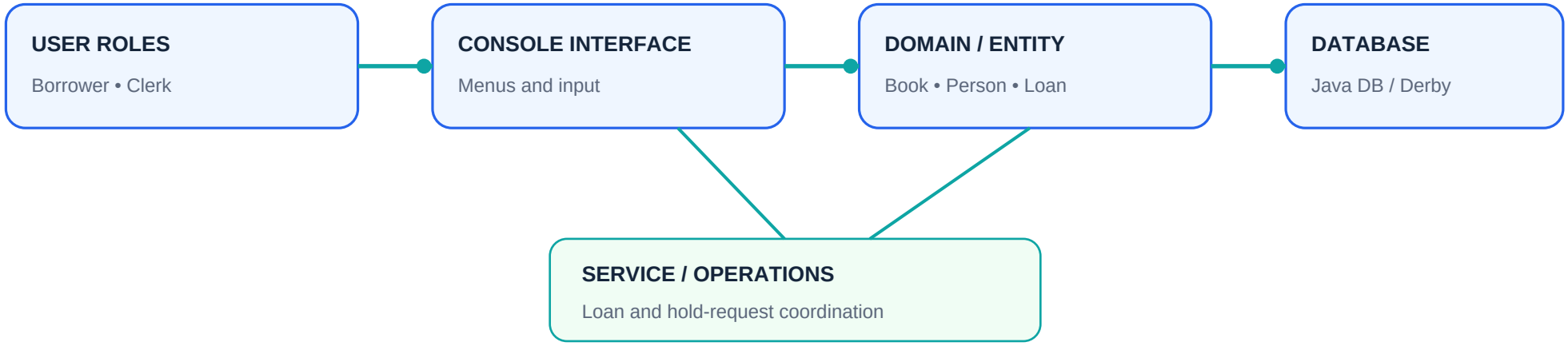
- Allow a borrower to place a request when a book is already on loan.

Administration

- Create staff accounts and review issued-book history and all books.

4. System Design and Architecture

The project emphasizes separation of responsibilities rather than placing all logic in the interface.



Interface layer Accepts user input and displays menu options.	**Domain entities** Represent people, books, staff, borrowers and loans.	**Operations** Coordinate tasks such as loan and hold-request processing.	**Persistence** Stores and retrieves records through the database connection.

Design note: the repository documents a refactoring that introduces *HoldRequestOperations* between HoldRequest and Book to reduce bidirectional dependency.

5. Technology Stack

The original repository describes a Java desktop/console application and a Java DB setup.

Java SE 8	Programming platform used to implement the application.
NetBeans IDE	Recommended IDE for opening and running the supplied project folder.
Java DB / Apache Derby	Relational database used to store library records.
JDBC	Database connectivity between Java code and the relational database.
StarUML	Can open the supplied .mdj class-diagram model.

Project assets: Java project folder, database folder, database schema text file, class diagram model and README documentation.

6. Database Design

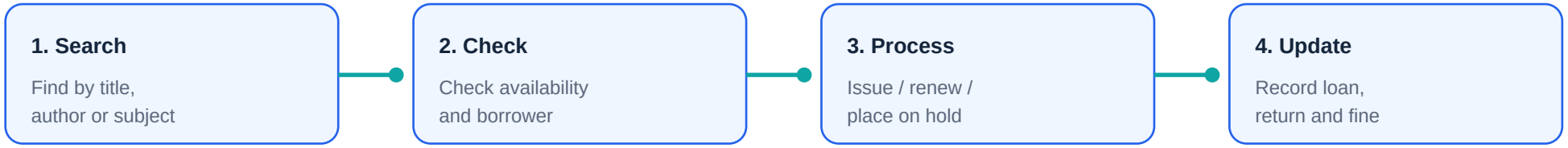
The supplied schema separates people, staff roles, books, borrowing and hold-request records.

Table	Purpose / key fields
PERSON	Common person details: ID, name, password, address and phone number.
BOOK	Book details: ID, title, author, subject and issued status.
STAFF / CLERK / LIBRARIAN	Staff type and role-specific information such as desk or office number.
BORROWER / BORROWED_BOOK	Borrower identity and book-to-borrower association.
LOAN	Borrower, book, issuing/receiving staff, issue date, return date and fine status.
ON_HOLD_BOOK	Hold request ID, book, borrower and request date.

Primary keys identify records; foreign-key relationships connect borrowers, staff, books and loan records.

7. Typical Book-Issue Workflow

A simplified view of how a borrower request can move through the system.



Issue

- Clerk selects the borrower and book.
- System records the loan and issue date.

Return

- Clerk checks in the book.
- Return date and receiving staff are recorded.

Renew / Hold

- A loan can be renewed.
- If a title is already on loan, a borrower can place a hold request.

The workflow is illustrative and summarizes the use cases documented in the repository.

8. Setup and Execution

The repository README provides the following environment setup sequence.

1. Install prerequisites	Install JDK 8 and NetBeans IDE.
2. Open the project	In NetBeans, choose File → Open Project and select the downloaded Project folder.
3. Configure Java DB	Set the Java DB database location to the repository's Database folder.
4. Create the connection	Create a Java DB Network connection for the LMS database using the connection details in the original README.
5. Connect and run	Connect to the database before running the project from NetBeans.

Note: This is the setup described by the original repository. Compatibility may vary with newer Java and NetBeans versions.

9. Benefits and Considerations

The design provides a useful learning example of object-oriented application development.

Benefits

- Organizes library tasks and records in one application.
- Uses separate entities and role-based responsibilities.
- Includes a documented database schema and class diagram.
- Covers core circulation tasks such as issue, return and renewal.

Considerations

- The documented interface is console-based, not a modern web UI.
- The repository setup targets JDK 8 and NetBeans Java DB.
- Security, usability and deployment should be reviewed before real-world use.
- The project may need maintenance for current development environments.

Possible future improvements: modern graphical/web interface, stronger authentication, automated fine calculation, reporting dashboards, and updated build/deployment tooling.

10. Conclusion and References

A concise summary of the system and the source material used for this presentation.

The Library Management System demonstrates how Java, object-oriented design and a relational database can support everyday library operations. Its role-based use cases cover searching, book collection maintenance, borrowing, returns, renewals, fines and hold requests.

> **Key takeaway**
>
> The main strength of the project is its structured separation of responsibilities: user roles, domain entities, operational logic and persistent database records.

Project repository
https://github.com/harismuneer/Library-Management-System-JAVA

Documentation consulted
README.md – project overview, actors, use cases and setup instructions.
Database Schema.txt – table definitions and foreign-key relationships.

Attribution
Original project by Haris Muneer and contributors. This presentation is an educational summary of the public repository; it does not claim authorship of the source code.

Thank You